Inherited-F4 replay determinism (root cause search), 2026-10-18

Status: **negative / partial.** The within-host sources of nondeterminism the review (`research/ic_leads_review_20261018` section 3.2) asked about are excluded by measurement and now pinned by a test. The cross-host drift itself is reproduced and localised (same source commit, different ISA, different counter) but **its cause was not found**: it needs an aarch64 run. Nothing here changes a counted quantity; the proposed accounting change is in "Proposal". Class: engineering (a regression test) plus a recorded null result.

The drift being explained

`research/ecbench_frontier_20261013` replay-all of session `ECBS1hfd5bf1885227` (Apple M4 Pro, aarch64, commit `2245b75`, rustc 1.93.1) on x86-64: 120/168 identical, all 48 inherited-F4 rows (arms `ic-f4-m2`, `ic-f4-m3`) differ in `relations.solver_ops` ("word XORs"). Only that counter differs in the record: `solver_calls`, `trials`, `relations`, and every `extra` counter (`splits`, `propagations`, `reductions`, `eliminated`, `infeasible_branches`) are equal, so the splitting trees are identical and only the word counts of the same trees differ. Example, seq 58 (`ic-f4-m2`): 300,531 (Apple) vs 301,961 (x86).

Reproduction (this container: x86-64, AMD EPYC 9V74, 4 vCPU, rustc

1.97.0) The spec `docs/ecbench/specs/claim-koblitz-rootfind-n13.json` (identical to the archived `spec.json` arms and seeds) was run three times with `--cpus none --allow-busy`, release at opt-level 1 (counts do not depend on optimisation), then audited with `ecbench verify --replay-all` under a different thread count.

Experiment	Result
RAYON_NUM_THREADS 1 vs 4 vs 4 (second process, new `RandomState` seeds)	48/48 F4 rows byte-identical `solver_ops` in all three sessions
`verify --replay-all` of the 4-thread session under 1 thread	OK, 168/168 reproduced
`verify --replay-all` of the 1-thread session under 4 threads	OK, 168/168 reproduced
`KIC_F4_EAGER=runs`, `KIC_F4_REWRITE=scatter`, `KIC_F4_EAGER=0` (the aarch64 code paths without BMI2 `pdep/pext`)	identical to default
A temporary switch forcing `wordwise_rewrite()` off (exact aarch64 configuration: eager shift runs + scatter rewrite)	identical to default
Salting `FxHasher`/`MaskHasher` (two nonzero salts, i.e. different table layouts and iteration orders)	identical to default
16 `KIC_F4_*`/`KIC_F6_*` settings, one at a time, on one `ic-f4-m2` row	identical to default
Archived Apple record vs x86, all 48 non-warm-up F4 rows	0/48 equal (`counters.json`, `counters.svg`)
The same, with x86 built from the **archived commit** `2245b75` (run during the search;	all 64 F4 rows (with warm-ups) differ from Apple, e.g. `ic-f4-m2` target 0 warm-up 417,827

numbers quoted from that run, the session was not kept)	(Apple) vs 419,828 (x86); `ic-f4-m3` 8,412,139 vs 10,305,435

The last row separates host from code drift: the archived commit on x86 already differs from the archived Apple record, and `ic-f4-m2` is unchanged between `2245b75` and current main on x86 (419,828 on both, checked on three rows). `ic-f4-m3` differs between the two x86 builds (10.3M vs 10.9M), so the current-binary replay mixes code change into the m3 gap; the ISA gap alone is the `2245b75`-on-x86 row.

Differences are always x86 > Apple, about 13 to 26 words per solver call for m2 (first 12 rows checked) and, over all 32 archived m2 rows including warm-ups, correlated with the number of propagations (r = 0.85, slope about 20 words per propagation; x86 `2245b75` against Apple), which points at the specialisation or completion charge rather than at elimination.

What is excluded, and what is left

Excluded on x86 (same binary, counters identical): thread count and rayon scheduling; `HashMap`/`HashSet` iteration order, both `RandomState` and the fixed-hash `FxMap`/`MaskMap` (hence also hashbrown's SSE2-vs-generic group width, which only changes layout); the BMI2 versus shift-run rewrite; the scatter versus plan rewrite; env overrides (the session's `child_env` contains only `RUST_*`).

Left, none testable here: (1) an aarch64-only difference in the engine, none located by reading `cfg(target_arch)` blocks in `inherited_f4.rs`, `pq_groebner_f2.rs`, `koblitz_groebner.rs` and `semaev_decomp.rs` (the x86 `pclmulqdq` fold and the table reduction are meant to give equal field values); (2) rustc 1.93.1 versus 1.97.0 library behaviour (for example `sort_unstable` on equal keys at `inherited_f4.rs` lines 557 and 2643); (3) macOS versus Linux; (4) a divergence in the archived Apple binary, whose `host.json` records `dirty: true` while `session.json` records `git_dirty: false`.

What landed

- `src/cryptanalysis/koblitz_groebner.rs`: test `inherited_f4_word_xors_ignore_thread_count_and_hash_salt` (24 random systems; rayon pools of 1 and 4 threads crossed with hash salts 0 and two nonzero values; the thread-local word counter must be equal and nonzero). `cargo test --lib inherited fx_hash`: 17 passed, 0 failed, 2 ignored.
- `src/cryptanalysis/fx_hash.rs`: a salt for the two monomial hashers that is a constant 0 outside `cfg(test)`, so production hashing and every count are unchanged. Before/after counters are therefore identical by construction; the table above is the measured confirmation.

Proposal (not applied)

Until an aarch64 run explains the gap, stage counts of the inherited-F4 arms should be treated as host-class-specific: version the method identity with the ISA (add `target_arch` to the `ic.pipeline` method fingerprint for `engine=inherited-f4`), or make the audit compare F4 `solver_ops` only within an `env_class`. Next measurement: run the committed spec on any aarch64 host with `KIC_F4_NODE_DUMP` (existing) and diff the per-node word counts against x86 to find the first diverging node.

Files

`counters.json` (recorded values, source of the graph), `plot.py`, `counters.svg`, `audits/` (two replay-all receipts), `REPORT.pdf`. Host of this work: x86-64, kernel 6.18.44, isolation L0 (operation counts

only). Session ids of the three re-runs: `ECBS1h76354bd30d5f`, `ECBS1h6b553bb18d3c`, `ECBS1hb29d912283e4` (not committed).

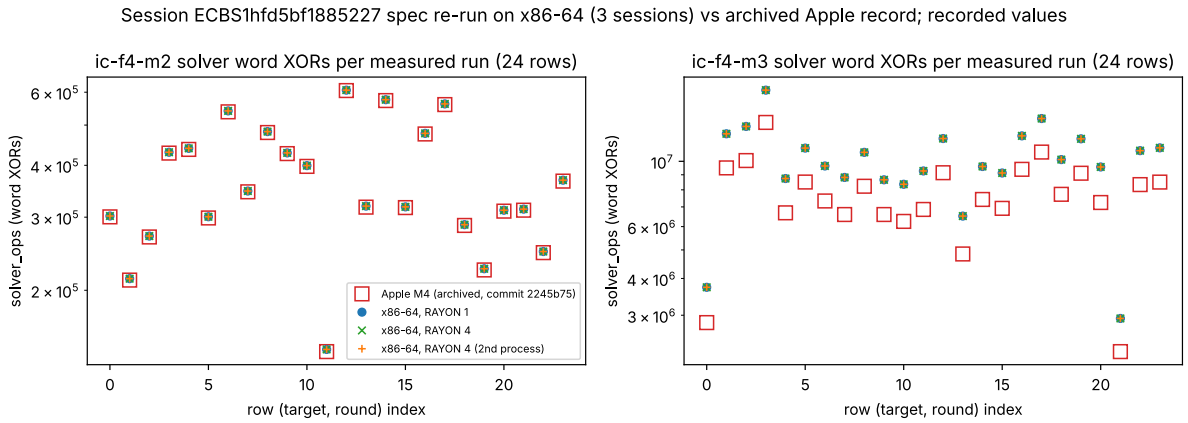


Figure 1: counters